

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.1
	Surname: Miranda Martín	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Mariana Patricia		



Activity 1. [Measuring execution times]

The idea is to perform an empirical study of the execution time of some programs.

We are going to use the Java method called `currentTimeMillis()` from the `System` class of the `java.lang` package (that is the only Java package that is preloaded, without the need for explicitly importing it using the `import` statement).

if we call this method twice, at the beginning and at the end of the measurement process, and if we subtract those two values, we will get the time spent between the calls in milliseconds.

Calculate how many more years we can continue using this way of counting. Explain what you did to calculate it.

How long can we use a 64-bit signed long to store milliseconds before it overflows?

A signed 64-bit long has a maximum value of $2^{63} - 1$, which is approximately 9,223,372,036,854,775,807 and there are approximately 31,557,600,000 milliseconds in a standard year.

Dividing the max value by ms/year gives approximately 292,277,026 years.

In conclusion, since its creation in 1970, we have about 292 million years remaining before `System.currentTimeMillis()` overflows, so no need to worry haha.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.1
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 2. [Measuring execution times]

Next, we will use the class called Vector2.java, which is a program that should be able to measure the time of an operation (algorithm), namely the addition of the n elements of a vector, that we have in the Vector1 class.

Why does the measured time sometimes come out as 0? From what size of problem (n) do we start to get reliable times

The measured time sometime come out as 0 because the `System.currentTimeMillis()` method has a limited resolution. If the algorithm is too simple and its execution is finished faster than this resolution, the difference between the start and end time is recorded as 0.

In order to avoid this problem and to get a reliable value, times above 50 ms are generally considered more accurate for individual measurements, i.e., we get reliable times from the problem size being between 200 000 and 300 000.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.1
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 3. [Taking small execution times (<50 ms)]

What happens with time if the problem size is multiplied by 2? What happens with time if the problem size is multiplied by a value k other than 2? (try it, for example, for $k=3$ and $k=4$ and check the times obtained)

Since the algorithm has a theoretical linear complexity $O(n)$, the time should increase by the same factor K and it does that indeed.

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.1
	Surname: Miranda Martín		
	Name: Mariana Patricia		

From what we saw in Vector4.java measuring the times for sum, create the following three Java classes:

- Vector5.java to measure times for maximum.
- Vector6.java to measure times for matches1.
- Vector7.java to measure times for matches2.

Indicate the main features (processor and memory) of the computer where times have been measured.

Laptop CPU and RAM – AMD Ryzen 7 5800H with Radeon Graphics, 16 GB

All the following tables are measured in milliseconds and WITHOUT OPTIMIZATION (-Xint) in Java. We can type “OoT” for times over one minute.

With the times obtained from the previous classes (in milliseconds), fill in the following two tables:

n	O(n)	O(n)
	Tsum	Tmaximum
10000	0,3	0,3
20000	0,5	0,5
40000	1,1	1,5
80000	2,1	2,7
16000	3,6	4,6
32000	6,8	9,3
64000	12	16,4
128000	27,6	32,3
256000	47,2	79,1
512000	102,8	151,9
1024000	244,1	320,5
2048000	478,9	617,2
4096000	943,5	1254,3
8192000	OoT	OoT

Algorithmics	Student information	Date	Number of session
	UO: 307423	26/02	1.1
	Surname: Miranda Martín		
	Name: Mariana Patricia		

NTIMES = 1	$O(n^2)$	$O(n)$
n	Tmatches1	Tmatches2
10000	1491	0
20000	6192	1
40000	28319	1
80000	102690	2
16000	OoT	3
32000	OoT	10
64000	OoT	14
128000	OoT	26
256000	OoT	60
512000	OoT	151
1024000	OoT	300
2048000	OoT	602
4096000	OoT	1049
8192000	OoT	OoT

Once both tables are filled in, conclude whether the times obtained meet what was expected, given the computational time complexity of the different operations.

Tsum and Tmaximum both follow a linear complexity, i.e., $O(n)$. As you can see Tmaximum is slightly slower because the CPU must perform a comparison and a potential update in every step, rather than just an addition.

Tmatches1 has a $O(n^2)$ complexity growth whereas Tmatches2 has a linear growth, i.e., $O(n)$, Tmatches2 must be significantly faster than the other one and the table above clearly showcases this.